

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Unit Tests
This test suite validates an AWS Glue PySpark ETL job that processes customer and order data. Here's a clear breakdown:---

****Test Structure Overview****

The code tests a data pipeline that:
1. Reads customer and order data from S3
2. Cleans the data (removes nulls/duplicates)
3. Joins and aggregates customer spending
4. Writes results to S3 and registers tables in AWS Glue Catalog---

****Key Components****

****1. Test Fixtures (Setup Data)****

****Spark Session Fixture****

```
@pytest.fixture(scope="session")def spark():
```

- Creates a local Spark session for testing- Runs once per test session- Cleans up after all tests complete
****Sample Data Fixtures****- `sample\_customer\_data`: 3 clean customer records- `sample\_order\_data`: 4 order records with prices and quantities- `sample\_customer\_data\_with\_nulls`: Customers with NULL values and "Null" strings- `sample\_customer\_data\_with\_duplicates`: Customers with duplicate records- `mock\_config`: Configuration dictionary with S3 paths, formats, and settings---

****2. TestConfigManager****

****Purpose****: Validates configuration loading from YAML files
****Test: `test\_get\_job\_parameters`****: Mocks reading a YAML configuration file- Verifies parameters are correctly parsed- Checks specific values like S3 paths and database names---

****3. TestS3DataReader****

****Purpose****: Tests reading data from S3 with validation
****Test: `test\_read\_data\_safe\_csv\_success`****: Mocks CSV reading from S3- Validates data is read correctly- Checks row count (3) and column names
****Test: `test\_read\_data\_safe\_schema\_validation\_failure`****: Tests schema validation when expected columns are missing- Verifies exception is raised with "Schema validation failed" message
****Test: `test\_read\_data\_safe\_empty\_path`****: Tests validation of empty S3 paths- Ensures ValueError is raised for invalid paths---

****4. TestS3DataWriter****

****Purpose****: Tests writing data to S3 in different formats
****Test: `test\_write\_data\_safe\_parquet`****: Mocks writing Parquet format- Verifies correct path and mode ("overwrite") are used
****Test: `test\_write\_data\_safe\_csv`****: Mocks writing CSV format- Verifies "append" mode is correctly applied---

****5. TestDataCleaner****

****Purpose****: Tests data quality operations
****Test: `test\_remove\_nulls\_and\_null\_strings`****: Input: 4 rows with NULLs and "Null" strings- Expected: 1 clean row (C001, John Doe)- Validates removal of both NULL values and literal "Null" strings
****Test: `test\_remove\_duplicates`****: Input: 4 rows with 1 duplicate-

Expected: 3 unique rows- Verifies deduplication logic**Test: `test\_clean\_dataframe`**- Tests complete cleaning pipeline (nulls + duplicates)- When enabled: removes problematic rows- When disabled: returns original data unchanged---

****6. TestGlueCatalogManager****

****Purpose****: Tests AWS Glue Catalog table registration****Test****: `test\_register\_table`**- Mocks Spark SQL operations- Verifies database creation SQL is executed- Verifies table drop SQL is executed (for overwrite)- Checks table write operation is called---

****7. TestDataAggregator****

****Purpose****: Tests business logic for calculating customer spending****Test****: `test\_calculate\_customer\_aggregate\_spend`**- Joins customer and order data- Calculates total spend per customer per date- ****Example calculation for C001 on 2024-01-01:**** - Order O001: $10.50 \times 2 = 21.00$ - Order O002: $25.00 \times 1 = 25.00$ - ****Total: 46.00****- Validates schema includes: custid, name, emailid, region, date, totalspend- Verifies calculation accuracy (within 0.01 tolerance)---

****8. TestMainFunction****

****Purpose****: Integration test for the entire ETL pipeline****Test****: `test\_main\_execution`**- Mocks all external dependencies (Spark, S3, Glue Catalog)- Simulates reading customer and order data- Verifies the complete workflow: 1. Reads customer data (1st call) 2. Reads order data (2nd call) 3. Writes customer table (1st write) 4. Writes order table (2nd write) 5. Writes aggregate table (3rd write)- Confirms all components are called in correct sequence---

****9. TestColumnNormalization****

****Purpose****: Tests column name standardization****Test****: `test\_column\_normalization`**- Input: Columns with mixed case (CustId, Name, EmailId, Region)- Expected: All lowercase (custid, name, emailid, region)- Ensures consistent column naming for downstream processing---

****Testing Patterns Used****

1. ****Mocking****: Uses `unittest.mock` to simulate S3 reads/writes without actual AWS calls2. ****Fixtures****: Reusable test data and configurations3. ****Assertions****: Validates counts, schemas, calculations, and error handling4. ****Isolation****: Each test is independent and doesn't affect others5. ****Coverage****: Tests happy paths, error cases, and edge conditions---

****Key Validations****

■ Data reading with schema validation ■ Data cleaning (nulls, duplicates, string normalization) ■ Business logic (spend aggregation with correct math) ■ Data writing in multiple formats ■ Glue Catalog integration ■ End-to-end pipeline execution ■ Error handling for invalid inputsThis comprehensive test suite ensures the ETL job is production-ready and handles real-world data quality issues.